

Object Oriented Programming

Prof. Dr. Muhammad Usman
Department of Computer Science
UET Mardan

Email: usman@uetmardan.edu.pk

SEPARATING INTERFACE FROM IMPLEMENTATION

Interfaces

- Interfaces define and standardize the ways in which things, such as people and systems, interact with one another
- The interface of a class describes **what services** clients of a class can use and **how to request** those services, but **not how the class carries out** these services
- A class's public interface consists of the class's public member functions

Separating the Interface from the Implementation

- So far, we wrote **class definition** and its **member function implementation** in **one file**
- However, it's better software engineering practice to define member functions outside the class definition, so that their implementation details can be hidden from the client.

SEPARATING INTERFACE FROM IMPLEMENTATION

- In this way the implementation will become separate from interface
- Changes in the implementation will not affect the clients as long as the public interface remains constant

Defining a Class's Interface with Function Prototypes

- The interface contains the **class definition** having data members and prototypes of member functions
- This *interface* will be saved as header file with **extension .h**
- This file tells the compiler about the functions name, its return types, the types of function parameters and class private data members because compiler must know the data members of the class to determine how much memory to reserve for each object of the class

SEPARATING INTERFACE FROM IMPLEMENTATION

Defining Member Functions in a Separate Source-Code File

- The definitions of the functions declared in **.h** file will be written in a **separate .cpp file**
- This file is the actual implementation file
- Changes in this file will not effect **.h** file as long as the private data members and function declaration remains the same
- Each member function name in the function headers in this file is preceded by the class name and **::** (binary scope resolution operator), otherwise the compiler will consider it a global function
- We must include the **.h** file in this **.cpp** file to indicate that these implementations belong to the class in **.h** file
- When compiling **.cpp** file the compiler uses the information in **.h** file to ensure that the **.cpp** file must match the prototypes in **.h** file and each member function knows about the class's data members and other member functions

SEPARATING INTERFACE FROM IMPLEMENTATION

Using the .h interface file

- To use the interface file we must include the **.h** file in our program
- Separating implementation from interface does not affect the way we write programs
- It affects only how the program is compiled and linked

The Compilation and Linking Process

- Often a class's interface and implementation are created and compiled by one programmer and used by a separate programmer who implements the client code that uses the class
- When the client code is compiled, the compiler uses the class definition in **.h** to ensure that the main function create and manipulate objects of class in **.h** file correctly

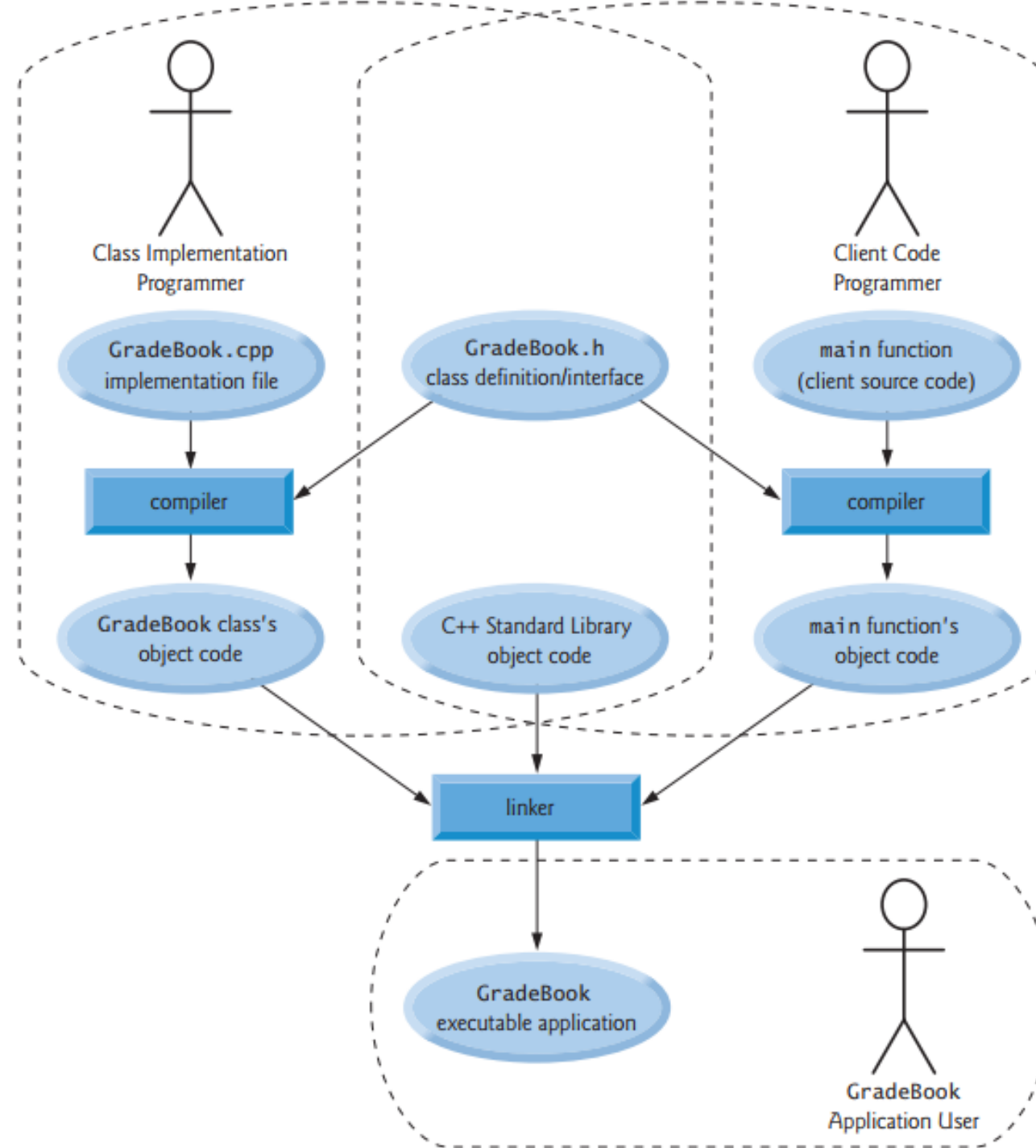
SEPARATING INTERFACE FROM IMPLEMENTATION

Header files

- Class definitions and function prototypes
- Included in each file using `#include`
- File extension `.h`

Source-code files

- Member function definitions
- Same base name
 - Convention
- Compiled and linked



SEPARATING INTERFACE FROM IMPLEMENTATION

Advantage

- Easier to modify programs

Disadvantage

- Header files contain
 - Portions of implementation
 - Inline member functions
 - Hints about other implementation
 - Private members

We Can hide more with **proxy** class


```

1. //time1.h
2. // Declaration of class Time.
3. // Member functions are defined
4. // prevent multiple inclusions of header
5.
6. #ifndef TIME1_H
7. #define TIME1_H
8.
9. // Time abstract data type definition
10. class Time {
11. public:
12.     Time();
13.     void setTime( int, int, int ); // set time
14.     void printUniversal(); // print universal-time format
15.     void printStandard(); // print standard-time format
16. private:
17.     int hour; // 0 - 23 (24-hour clock format)
18.     int minute; // 0 - 59
19.     int second; // 0 - 59
20. }; // end class Time
21.
22.
23.
24.
25. #endif

```

Preprocessor code to prevent multiple inclusions.

Code between these directives

Preprocessor directive defines name **TIME1_H**.

"If not defined"

Naming convention: header file name with underscore replacing period.

EXAMPLE

#ifndef stands for "**if not defined**". It is a **preprocessor directive** used to prevent a header file from being included **more than once** in the same compilation — this is called an **include guard**.

```
1 // time1.cpp
2 // Member-function definitions for class Time.
3 #include <iostream>
4
5 using std::cout;
6
7 #include <iomanip>
8
9 using std::setfill;
10 using std::setw;
11
12 // include definition of class Time from time1.h
13 #include "time1.h"
14
15 // Time constructor initializes
16 // Ensures all Time objects start at 0.
17 Time::Time()
18 {
19     hour = minute = second = 0;
20
21 } // end Time constructor
22
```

Include header file
time1.h.

Name of header file enclosed
in quotes; angle brackets
cause preprocessor to assume
header part of C++ Standard
Library.

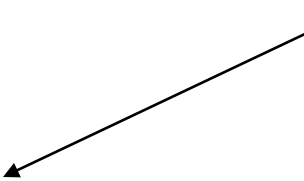
```
23 // Set new Time value using universal time. Perform validity
24 // checks on the data values. Set invalid values to zero.
25 void Time::setTime( int h, int m, int s )
26 {
27     hour = ( h >= 0 && h < 24 ) ? h : 0;
28     minute = ( m >= 0 && m < 60 ) ? m : 0;
29     second = ( s >= 0 && s < 60 ) ? s : 0;
30
31 } // end function setTime
32
33 // print Time in universal format
34 void Time::printUniversal()
35 {
36     cout << setfill( '0' ) << setw( 2 ) << hour << ":"
37         << setw( 2 ) << minute << ":"
38         << setw( 2 ) << second;
39
40 } // end function printUniversal
41
```

```
42 // print Time in standard format
43 void Time::printStandard()
44 {
45     cout << ( ( hour == 0 || hour == 12 ) ? 12 : hour % 12 )
46         << ":" << setfill( '0' ) << setw( 2 ) << minute
47         << ":" << setw( 2 ) << second
48         << ( hour < 12 ? " AM" : " PM" );
49
50 } // end function printStandard
```

MAINPROG.CPP (1 OF 2)

```
1  // mainprog.cpp
2  // Program to test class Time.
3  // NOTE: This file must be compiled with time1.cpp.
4  #include <iostream>
5
6  using std::cout;
7  using std::endl;
8
9  // include definition of class Time from time1.h
10 #include "time1.h"
11
12 int main()
13 {
14     Time t; // instantiate object t of class Time
15
16     // output Time object t's initial values
17     cout << "The initial universal time is ";
18     t.printUniversal(); // 00:00:00
19     cout << "\nThe initial standard time is ";
20     t.printStandard(); // 12:00:00 AM
21
22     t.setTime( 13, 27, 6 ); // change time
23
```

Include header file **time1.h** to ensure correct creation/manipulation and determine size of **Time** class object.



```
24 // output Time object t's new values
25 cout << "\n\nUniversal time after setTime is ";
26 t.printUniversal(); // 13:27:06
27 cout << "\nStandard time after setTime is ";
28 t.printStandard(); // 1:27:06 PM
29
30 t.setTime( 99, 99, 99 ); // attempt invalid settings
31
32 // output t's values after specifying invalid values
33 cout << "\n\nAfter attempting invalid settings:"
34     << "\nUniversal time: ";
35 t.printUniversal(); // 00:00:00
36 cout << "\nStandard time: ";
37 t.printStandard(); // 12:00:00 AM
38 cout << endl;
39
40 return 0;
41
42 } // end main
```

```
The initial universal time is 00:00:00
The initial standard time is 12:00:00 AM

Universal time after setTime is 13:27:06
Standard time after setTime is 1:27:06 PM
```

```
#ifndef FILENAME_H
```

```
#define FILENAME_H
```

```
// your header file content here
```

```
#endif
```

If you include the same header file multiple **times** (directly or indirectly), the compiler will see duplicate declarations and **throw** an error. `#ifndef` prevents **this**.

`#ifndef` stands for **"if not defined"**.

It is a **preprocessor directive** used to prevent a header file from being included **more than once** in the same compilation — this is called an **include guard**.

```
**Without guard – Problem:**  
...
```

```
mainprog.cpp
```

```
└─ includes interface.h      ✓ first time – OK
```

```
└─ includes interfdef.cpp
```

```
    └─ includes interface.h again ✗ DUPLICATE – ERROR
```

```
...
```

```
**With guard – Safe:**
```

```
...
```

```
mainprog.cpp
```

```
└─ includes interface.h      ✓ first time – defined
```

```
└─ includes interfdef.cpp
```

```
    └─ includes interface.h again – skipped ✓
```

CONST (CONSTANT) OBJECTS AND CONST MEMBER FUNCTIONS

- Some objects need to be modifiable and some do not
- Principle of least privilege
 - Only allow modification of necessary objects
- The programmer may use the keyword `const` to specify that an object is not modifiable
 - Any attempt to modify the Const object is a **syntax error**

▪ Example

```
const Time noon( 12, 0, 0 );
```

- Declares const object noon of class Time and Initializes to 12

CONST (CONSTANT) OBJECTS AND CONST MEMBER FUNCTIONS

- Member functions for **const** objects must also be **const**
 - Cannot modify object
 - Member functions declared **const** can not modify the object, the compiler disallows this
- Specify **const** in both prototype and definition of member function
 - Prototype
 - Const After parameter list
 - Definition
 - Const Before beginning left brace(i.e., after the function parameter list)

const Member Function

- ✗ A **const** member function guarantees that it will never modify any of its class's member data.

```
//constfu.cpp demonstrates const member functions
class aClass
{
private:
    int alpha;
public:
    void nonFunc()                // non-const member function
    {
        alpha = 99;              // OK
    }

    void conFunc() const          // const member function
    {
        alpha = 99;              // ERROR: can't modify a member
    }
};
```

Distance Class and Use of **const**

(1/3)

```
// const member functions & const arguments to member functions
#include <iostream>
using namespace std;

class Distance                                // English Distance class
{
private:
    int feet;
    float inches;
public:
    Distance() : feet(0), inches(0.0)         // constructor (no args)
    { }                                       // constructor (two args)

    Distance(int ft, float in) : feet(ft), inches(in)
    { }

    void getdist(){                          // get length from user
        cout << "\nEnter feet: "; cin >> feet;
        cout << "Enter inches: "; cin >> inches;
    }
}
```

Distance Class and Use of `const`

(2/3)

```
void showdist() const           // display distance
{ cout << feet << "\'-" << inches << '\\"'; }
Distance add_dist(const Distance&) const; // add
};

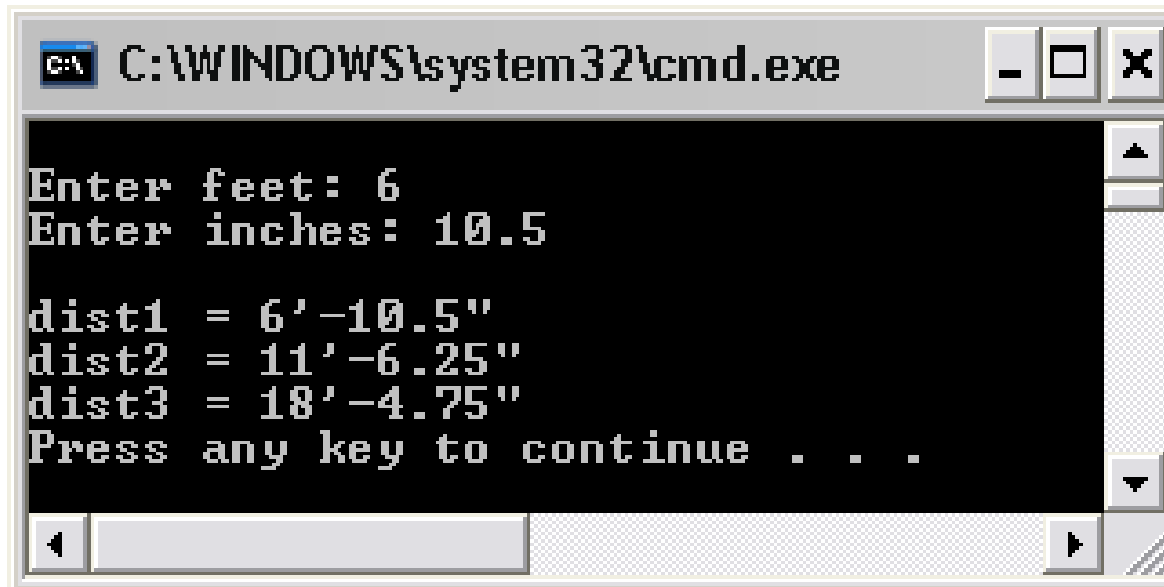
//add this distance to d2, return the sum
Distance Distance::add_dist(const Distance& d2) const
{
    Distance temp;                // temporary variable
    // feet = 0;                  // ERROR: can't modify this
    // d2.feet = 0;               // ERROR: can't modify d2
    temp.inches = inches + d2.inches; // add the inches
    if(temp.inches >= 12.0)          // if total exceeds 12.0,
    { temp.inches -= 12.0;           // then decrease inches by 12.0
      temp.feet = 1;                // and increase feet by 1
    }
    temp.feet += feet + d2.feet;    // add the feet
    return temp;                    // return temp object
}
```

Distance Class and Use of `const`

(3/3)

```
int main()
{
    Distance dist1, dist3;           // define two lengths
    Distance dist2(11, 6.25);        // define, initialize dist2
    dist1.getdist();                  // get dist1 from user
    dist3 = dist1.add_dist(dist2);    // dist3 = dist1 + dist2
    //display all lengths
    cout << "\ndist1 = "; dist1.showdist();
    cout << "\ndist2 = "; dist2.showdist();
    cout << "\ndist3 = "; dist3.showdist();
    cout << endl;

    return 0;
}
```



```
C:\WINDOWS\system32\cmd.exe

Enter feet: 6
Enter inches: 10.5

dist1 = 6'-10.5"
dist2 = 11'-6.25"
dist3 = 18'-4.75"
Press any key to continue . . .
```

const Member Function Arguments

- ✖ if an argument is passed to an **ordinary function** by reference, and **you don't want the function to modify it**, the argument should be made `const` in the function declaration (and definition). This is true of member functions as well.
`Distance Distance::add_dist(const Distance& d2) const{`
- ✖ In above line, argument to `add_dist()` is passed by reference, and we want to make sure that it won't modify this variable, which is `dist2` in `main()`.
- ✖ Therefore we make the argument `d2` to `add_dist()` `const` in both declaration and definition.

const Objects

- ✖ As we can apply `const` to variables of basic types such as `int` to keep them from being modified.
- ✖ In a similar way, we can apply `const` to `objects` of `classes`. When an `object` is declared as `const`, `you can't modify it`.
- ✖ It follows that `you can use only const member functions with it`, because they're the only ones that guarantee not to modify it. e.g. A football field (for American-style football) is exactly 300 feet long. If we were to use the length of a football field in a program, it would make sense to make it `const`, because changing it would represent the end of the world for football fans.

const Objects

(1/2)

```
// constObj.cpp constant Distance objects
#include <iostream>
using namespace std;
class Distance                                // English Distance class
{
private:
    int feet;
    float inches;
public:
    // 2-arg constructor
    Distance(int ft, float in) : feet(ft), inches(in)
    { }

    void getdist()                            // user input; non-const function
    {
        cout << "\nEnter feet:" ; cin >> feet;
        cout << "Enter inches:" ; cin >> inches;
    }
}
```


const Objects

(2/2)

```
void showdist() const           // display distance; const function
{
    cout << feet << "\'-" << inches << '\\"';
}
};

int main()
{
    const Distance football(300, 0);

    // football.getdist();    // ERROR: getdist() not const

    cout << "football = " ;
    football.showdist();      // OK
    cout << endl;
    return 0;
}
```

```
1 // time5.h
2 // Definition of class Time.
3 // Member functions defined in time5.cpp.
4 #ifndef TIME5_H
5 #define TIME5_H
6
7 class Time {
8
9 public:
10     Time( int = 0, int = 0, int = 0 ); // default constructor
11
12     // set functions
13     void setTime( int, int, int ); // set time
14     void setHour( int ); // set hour
15     void setMinute( int ); // set minute
16     void setSecond( int ); // set second
17
18     // get functions (normally declared const)
19     int getHour() const; // return hour
20     int getMinute() const; // return minute
21     int getSecond() const; // return second
```

ANOTHER EXAMPLE

Declare **const** get functions.

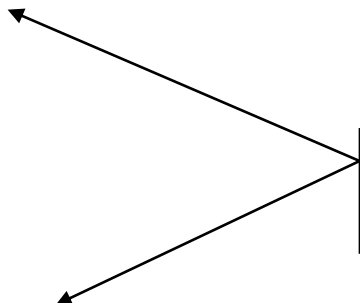
```
22
23 // print functions (normally declared const)
24 void printUniversal() const; // print universal time
25 void printStandard(); // print standard time
26
27 private:
28     int hour; // 0 - 23 (24-hour clock format)
29     int minute; // 0 - 59
30     int second; // 0 - 59
31
32 }; // end class Time
33
34 #endif
```

Declare **const** function
printUniversal.

```
1  // time5.cpp
2  // Member-function definitions for class Time.
3  #include <iostream>
4
5  using std::cout;
6
7  #include <iomanip>
8
9  using std::setfill;
10 using std::setw;
11
12 // include definition of class Time from time5.h
13 #include "time5.h"
14
15 // constructor function to initialize private data;
16 // calls member function setTime to set variables;
17 // default values are 0 (see class definition)
18 Time::Time( int hour, int minute, int second )
19 {
20     setTime( hour, minute, second );
21
22 } // end Time constructor
```

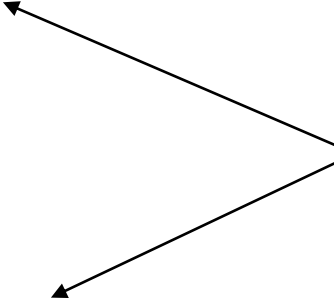
```
24 // set hour, minute and second values
25 void Time::setTime( int hour, int minute, int second )
26 {
27     setHour( hour );
28     setMinute( minute );
29     setSecond( second );
30
31 } // end function setTime
32
33 // set hour value
34 void Time::setHour( int h )
35 {
36     hour = ( h >= 0 && h < 24 ) ? h : 0;
37
38 } // end function setHour
39
40 // set minute value
41 void Time::setMinute( int m )
42 {
43     minute = ( m >= 0 && m < 60 ) ? m : 0;
44
45 } // end function setMinute
```

```
47 // set second value
48 void Time::setSecond( int s )
49 {
50     second = ( s >= 0 && s < 60 ) ? s : 0;
51
52 } // end function setSecond
53
54 // return hour value
55 int Time::getHour() const
56 {
57     return hour;
58
59 } // end function getHour
60
61 // return minute value
62 int Time::getMinute() const
63 {
64     return minute;
65
66 } // end function getMinute
67
```



const functions do not
modify objects.

```
68 // return second value
69 int Time::getSecond() const
70 {
71     return second;
72 }
73 // end function getSecond
74
75 // print Time in universal format
76 void Time::printUniversal() const
77 {
78     cout << setfill( '0' ) << setw( 2 ) << hour << ":"
79         << setw( 2 ) << minute << ":"
80         << setw( 2 ) << second;
81 }
82 // end function printUniversal
83
84 // print Time in standard format
85 void Time::printStandard() // note lack of const declaration
86 {
87     cout << ( ( hour == 0 || hour == 12 ) ? 12 : hour % 12 )
88         << ":" << setfill( '0' ) << setw( 2 ) << minute
89         << ":" << setw( 2 ) << second
90         << ( hour < 12 ? " AM" : " PM" );
91 }
92 // end function printStandard
```



const functions do not
modify objects.

```
1 // Fig. 10.3: fig10_03.cpp
2 // Attempting to access a const object with non-const member functions.
3 #include "Time.h" // include Time class definition
4
5 int main()
6 {
7     Time wakeUp( 6, 45, 0 ); // non-constant object
8     const Time noon( 12, 0, 0 ); // constant object
9
10    // OBJECT MEMBER FUNCTION
11    wakeUp.setHour( 18 ); // non-const non-const
12
13    noon.setHour( 12 ); // const non-const
14
15    wakeUp.getHour(); // non-const const
16
17    noon.getMinute(); // const const
18    noon.printUniversal(); // const const
19
20    noon.printStandard(); // const non-const
21 } //endmain
```

Note that non-**const** constructor can initialize **const** object.

Declare **noon** a **const** object.

CONST (CONSTANT) OBJECTS AND CONST MEMBER FUNCTIONS

const with Constructor and Destructor

- Constructor and destructors can not be const
- A constructor must be allowed to modify an object so that the object can be initialized properly
- A destructor must be able to perform its termination housekeeping before an object is destroyed
- A constructor must be a non-const member function, it can still be called for a const object

CONST VS NON-CONST MEMBER FUNCTIONS

Member initializer Syntax

- Initializing with member initializer syntax
 - can be used for all data members
 - must be used for
 - `const` data members
 - Data members that are references

```
// constructor
Time::Time(int h, int m, int s)
    :hour( h ),    // initializer for non-const member
    minute( m ),  // required initializer for const member
    second( s )   // required initializer for const member
{ }
```

```
1  // membinit.cpp
2  // Using a member initializer to initialize a
3  // constant of a built-in data type.
4  #include <iostream>
5
6  using std::cout;
7  using std::endl;
8
9  class Increment {
10
11  public:
12      Increment( int c = 0, int i = 1 ); // default constructor
13
14      void addIncrement()
15      {
16          count += increment;
17
18      } // end function addIncrement
19
20      void print() const; // prints count and increment
21
```

```

22 private:
23     int count;
24     const int increment; // const data member
25
26 }; // end class Increment
27
28 // constructor
29 Increment::Increment(int c, int i)
30 : count( c ), // initialize member count.
31   increment( i ) // require member increment
32 {
33     // empty body
34
35 } // end Increment constructor
36
37 // print count and increment
38 void Increment::print() const
39 {
40     cout << "count = " << count
41          << ", increment = " << increment << endl;
42
43 } // end function print

```

Declare increment as **const** data member.

Member initializer list separated from parameter list by colon.

Member initializer syntax can be used for non-**const** data member **count**.

Member initializer syntax must be used for **const** data member **increment**.

Member initializer consists of data member name (**increment**) followed by parentheses containing initial value (**c**).

```
45  int main()
46  {
47      Increment value( 10, 5 );
48
49      cout << "Before incrementing: ";
50      value.print();
51
52      for ( int i = 0; i < 3; i++ ) {
53          value.addIncrement();
54          cout << "After increment " << i + 1 << ": ";
55          value.print();
56      }
57
58      return 0;
59
60  } // end main
```

```
Before incrementing: count = 10, increment = 5
After increment 1: count = 15, increment = 5
After increment 2: count = 20, increment = 5
After increment 3: count = 25, increment = 5
```